Computability – Topic 7

(with Recitation 8 Material)

Yonghao Lee

February 2026

Contents

1 Reducibility **2**
1.1 HALT_{TM} is Undecidable . . . 2
1.2 E_{TM} is Undecidable . . . 3
1.3 $\text{REGULAR}_{\text{TM}}$ is Undecidable . . . 4
1.4 Comparing the Three Proofs . . . 6

2 Mapping Reducibility **6**
2.1 Computable Functions . . . 6
2.2 Formal Definition . . . 6
2.3 The Reduction Theorem . . . 7
2.4 The Complement Trick . . . 8
2.5 The Three-Part Proof Template . . . 8

3 Language Classification Examples **8**
3.1 A Decidable Example: Time-Bounded Machines . . . 8
3.2 $\text{NON-EMPTY}_{\text{TM}} \in \text{RE} \setminus \text{R}$ . . . 9
3.3 $\text{EMPTY}_{\text{TM}} \in \text{co-RE} \setminus \text{R}$ . . . 10
3.4 $\text{ALL}_{\text{TM}} \notin \text{RE} \cup \text{co-RE}$ . . . 10
3.5 EQ_{TM}: Undecidable and Neither RE nor co-RE . . . 11
3.6 Every RE Language Reduces to A_{TM} . . . 12

4 Exam-Style Examples **12**
4.1 Machines That Accept Every Word of Some Length . . . 12
4.2 Machines with Reverse-Equal Languages . . . 13

5 Summary **13**
5.1 Classification Table . . . 13
5.2 Reduction Patterns . . . 14

1 Reducibility

We have presented several examples of problems that are solvable on a TM and one example — A_{TM} — that is computationally unsolvable. Now, we examine several additional unsolvable problems. In doing so, we introduce the primary method for proving that problems are computationally unsolvable: **reducibility**.

A **reduction** is a way of converting one problem to another problem in such a way that a solution to the second problem can be used to solve the first problem. Reducibility always involves two problems, which we call A and B. If A reduces to B, we can use a solution to B to solve A.

Reducibility plays an important role in classifying problems by decidability. When A is reducible to B, solving A cannot be harder than solving B, because a solution to B gives a solution to A. In terms of computability theory:

- If A is reducible to B and B is decidable, then A is also decidable.
- Equivalently, if A is undecidable and reducible to B, then B is undecidable.

In short, our method for proving that a problem is undecidable will be to show that some other problem already known to be undecidable reduces to it.

In this section we illustrate the idea with three "textbook" undecidability proofs, all reducing from A_{TM}. The formal definition of mapping reducibility comes in Section 2; the proofs here use the informal "assume a decider, build a contradiction" style.

1.1 HALT_{TM} is Undecidable

Definition 1.1: The Halting Problem

$$\text{HALT}_{\text{TM}} = \{\langle M, w\rangle \mid M \text{ is a TM and } M \text{ halts on input } w\}$$

Theorem 1.1: HALT_{TM} is Undecidable

HALT_{TM} is undecidable.

The proof is by contradiction. We assume that HALT_{TM} is decidable and use that assumption to show that A_{TM} is decidable, contradicting what we have already established. The key idea is to show that A_{TM} is reducible to HALT_{TM}.

Suppose we have a TM R that decides HALT_{TM}. We use R to construct S, a TM that decides A_{TM}. Given an input of the form $\langle M, w\rangle$, S must accept if M accepts w, and reject if M loops or rejects w.

With the help of R, we can first test whether M halts on w:

- If R indicates that M does *not* halt on w, then we immediately reject, because $\langle M, w\rangle \notin A_{\text{TM}}$.
- If R indicates that M *does* halt on w, we can safely simulate M on w without any danger of looping.

Thus, if such a TM R exists, we can decide A_{TM}. But we know that A_{TM} is undecidable. Therefore R does not exist, and HALT_{TM} is undecidable.

Proof. We construct a TM S that decides A_{TM} using R as a subroutine.

TM S deciding A_{TM} (assuming R decides HALT_{TM})

S = "On input $\langle M, w\rangle$, where M is a TM and w is a string:
1. Run TM R on input $\langle M, w\rangle$.
2. If R rejects (i.e., M does not halt on w), **reject**.
3. If R accepts (i.e., M halts on w), simulate M on w until it halts.
4. If M accepted, **accept**. If M rejected, **reject**."

If R decides HALT_{TM}, then S decides A_{TM}: in step 3, the simulation is guaranteed to terminate because R has confirmed that M halts on w. Since A_{TM} is undecidable, the decider R for HALT_{TM} cannot exist. Therefore HALT_{TM} is undecidable. □

Remark 1.1: Structure of an Informal Reduction Proof

Notice the general pattern:

(i) **Assume** the target problem (HALT_{TM}) is decidable via some TM R.
(ii) **Construct** a decider for a known undecidable problem (A_{TM}) using R as a subroutine.
(iii) **Conclude** that R cannot exist, since the known problem is undecidable.

This is the template we will use repeatedly. In Section 2 we will see the formal version — mapping reducibility — which replaces the "assume a decider" argument with an explicit computable function.

1.2 E_{TM} is Undecidable

Definition 1.2: E_{TM}

$$E_{\text{TM}} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset\}$$

Theorem 1.2: E_{TM} is Undecidable

E_{TM} is undecidable.

The idea is to reduce from A_{TM} again. Suppose R is a TM that decides E_{TM}. We use R to construct a TM S that decides A_{TM}.

How should S work when it receives input $\langle M, w \rangle$? A first idea is to run R on input $\langle M \rangle$ and see whether it accepts. If it does, we know that $L(M) = \emptyset$, so M does not accept w. But if R rejects $\langle M \rangle$, all we know is that $L(M) \neq \emptyset$ — that is, M accepts *some* string. We still do not know whether M accepts the particular string w. So we need a different approach.

Instead of running R on $\langle M \rangle$ directly, we run R on a **modified** version of M. We modify M to guarantee that it rejects all strings except w, but on input w it behaves as usual. Then we use R to determine whether this modified machine recognizes the empty language. The only string the modified machine can possibly accept is w, so its language will be nonempty if and only if it accepts w.

- If R accepts the description of the modified machine, its language is empty, meaning M does not accept w.
- If R rejects the description of the modified machine, its language is nonempty, meaning M accepts w.

Proof. Assume that TM R decides E_{TM}. We construct TM S that decides A_{TM}.

TM S deciding A_{TM} (assuming R decides E_{TM})

S = "On input $\langle M, w \rangle$, where M is a TM and w is a string:

1. Use the descriptions of M and w to construct the following TM M_1:

   TM M_1

   M_1 = "On input x:
   1. If $x \neq w$, **reject**.
   2. If $x = w$, run M on input w and **accept** if M does."

2. Run R on input $\langle M_1 \rangle$.
3. If R accepts (i.e., $L(M_1) = \emptyset$, so M does not accept w), **reject**.
4. If R rejects (i.e., $L(M_1) \neq \emptyset$, so M accepts w), **accept**."

The machine M_1 has the string w as part of its description. It tests whether $x = w$ by scanning and comparing characters one by one. Note that S can compute the description $\langle M_1 \rangle$ from $\langle M \rangle$ and w: it only needs to add extra states to M that perform the $x = w$ comparison.

By construction, $L(M_1) = \emptyset$ if and only if M does not accept w. Therefore, if R decides E_{TM}, then S decides A_{TM}. Since A_{TM} is undecidable, the decider R for E_{TM} cannot exist. Therefore E_{TM} is undecidable. □

Remark 1.2: The Modification Technique

This proof introduces an important technique: rather than feeding R the original machine $\langle M \rangle$, we construct a **helper machine** M_1 that is tailor-made to encode the question "does M accept w?" into a form that R can answer. This idea of building a custom machine inside a reduction proof is a recurring pattern in undecidability arguments.

1.3 $\text{REGULAR}_{\text{TM}}$ is Undecidable

Another interesting computational problem concerning TMs involves determining whether a given TM recognizes a language that can also be recognized by a simpler model. We let $\text{REGULAR}_{\text{TM}}$ be the problem of determining whether a given TM has an equivalent finite automaton — equivalently, whether the TM recognizes a regular language.

Definition 1.3: $\text{REGULAR}_{\text{TM}}$

$$\text{REGULAR}_{\text{TM}} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) \text{ is a regular language}\}$$

Theorem 1.3: $\text{REGULAR}_{\text{TM}}$ **is Undecidable**

$\text{REGULAR}_{\text{TM}}$ is undecidable.

The idea is to reduce from A_{TM} again. We assume that $\text{REGULAR}_{\text{TM}}$ is decidable by some TM R and use this assumption to construct a TM S that decides A_{TM}.

Less obvious now is how to use R's ability to assist S. The idea is for S to take its input $\langle M, w \rangle$ and modify M into a new machine M_2 whose language is **controlled** by whether M accepts w:

- If M accepts w, then $L(M_2)$ should be some **regular** language.
- If M does not accept w, then $L(M_2)$ should be some **non-regular** language.

Then R, which can supposedly distinguish regular from non-regular, will indirectly tell us whether M accepts w.

Understanding the Construction of M_2

The machine M_2 is built as follows:

TM M_2

$M_2 =$ "On input x:
1. If x has the form $0^n 1^n$ for some $n \geq 0$, **accept**.
2. Otherwise, run M on input w and **accept** if M does."

This may look strange at first. Let us carefully understand what M_2 does and *why* it works.

Key observation: Step 2 **ignores** the input x entirely. It does not run M on x — it runs M on the *fixed* string w. The input x only matters in step 1, where we check if $x \in \{0^n 1^n\}$.

Case analysis: Consider what $L(M_2)$ looks like in each case.

- **Case 1: M accepts w.**
  - If $x = 0^n1^n$: step 1 accepts. ✓
  - If $x \neq 0^n1^n$: step 2 runs M on w. Since M accepts w, step 2 accepts. ✓

  Every string is accepted, so $L(M_2) = \Sigma^*$. This is a **regular** language.

- **Case 2: M does not accept w.**
  - If $x = 0^n1^n$: step 1 accepts. ✓
  - If $x \neq 0^n1^n$: step 2 runs M on w. Since M does not accept w, step 2 does not accept (M either rejects or loops). ✗

  Only strings of the form 0^n1^n are accepted, so $L(M_2) = \{0^n1^n \mid n \geq 0\}$. This is **not regular** (by the pumping lemma).

Does M accept w?	$L(M_2)$	**Regular?**
Yes	Σ^*	✓
No	$\{0^n1^n \mid n \geq 0\}$	✗

Why 0^n1^n?

The choice of $\{0^n1^n\}$ is not magical — any non-regular language would work. We just need *some* language that is not regular to serve as the "default" behavior. The construction guarantees that:

- M_2 always accepts $\{0^n1^n\}$ as a "baseline" (step 1).
- Step 2 acts as a "switch": if M accepts w, the switch turns on and M_2 accepts *everything else* too, inflating the language to Σ^*.

The result is that $L(M_2)$ toggles between a non-regular language and a regular one, controlled entirely by whether M accepts w.

The Full Proof

Proof. Assume that TM R decides $\text{REGULAR}_{\text{TM}}$. We construct TM S that decides A_{TM}.

TM S deciding A_{TM} (assuming R decides $\text{REGULAR}_{\text{TM}}$)

S = "On input $\langle M, w\rangle$, where M is a TM and w is a string:
1. Use the descriptions of M and w to construct TM M_2 as described above.
2. Run R on input $\langle M_2\rangle$.
3. If R accepts (i.e., $L(M_2)$ is regular, so M accepts w), **accept**.
4. If R rejects (i.e., $L(M_2)$ is not regular, so M does not accept w), **reject**."

By the case analysis above, $L(M_2)$ is regular if and only if M accepts w. Therefore, if R decides $\text{REGULAR}_{\text{TM}}$, then S decides A_{TM}. Since A_{TM} is undecidable, the decider R for $\text{REGULAR}_{\text{TM}}$ cannot exist. Therefore $\text{REGULAR}_{\text{TM}}$ is undecidable. □

1.4 Comparing the Three Proofs

Remark 1.3: Comparing the Three Reduction Proofs

All three undecidability proofs follow the same template: assume a decider R exists for the target problem, then use R to build a decider S for A_{TM}, reaching a contradiction. The key difference lies in how the helper machine is constructed:

Problem	**Helper TM**	M **accepts** w	M **doesn't accept** w
$\mathrm{HALT}_{\mathrm{TM}}$	(direct simulation)	M halts on w	M may not halt
E_{TM}	M_1	$L(M_1) = \{w\} \neq \emptyset$	$L(M_1) = \emptyset$
$\mathrm{REGULAR}_{\mathrm{TM}}$	M_2	$L(M_2) = \Sigma^*$ (regular)	$L(M_2) = \{0^n 1^n\}$ (not regular)

In each case, the helper machine translates the question "does M accept w?" into a question that R can supposedly answer. The recipe is always the same:

(i) Build a helper machine whose **language property** is controlled by whether M accepts w.
(ii) Use the assumed decider R to test that property.
(iii) R's answer directly tells you whether M accepts w, so you have decided A_{TM} — a contradiction.

Remark 1.4: Rice's Theorem (Preview)

The undecidability of E_{TM} and $\mathrm{REGULAR}_{\mathrm{TM}}$ are not isolated results. Similarly, testing whether the language of a TM is context-free, decidable, finite, or has any other nontrivial property can be shown undecidable with analogous proofs. This is captured by **Rice's theorem**, which states that *any* nontrivial property of the languages recognized by Turing machines is undecidable. (A property is "nontrivial" if some TMs have it and some do not.)

2 Mapping Reducibility

The proofs in Section 1 all used the same informal strategy: assume a decider exists, use it as a subroutine to decide A_{TM}, and derive a contradiction. In this section, we formalize the notion of reducibility itself, which makes the reasoning more precise and more powerful — in particular, it lets us prove statements about RE and co-RE, not just decidability.

2.1 Computable Functions

A TM computes a function by starting with the input to the function on the tape and halting with the output of the function on the tape.

Definition 2.1: Computable Function

A function $f\colon \Sigma^* \to \Sigma^*$ is a **computable function** if some TM M, on every input w, halts with just $f(w)$ on its tape.

2.2 Formal Definition

Definition 2.2: Mapping Reducibility

A language A is **mapping reducible** to language B, written $A \leq_{\mathrm{m}} B$, if there is a computable function $f\colon \Sigma^* \to \Sigma^*$ such that for every w:

$$w \in A \iff f(w) \in B$$

The function f is called the **reduction** from A to B.

Remark 2.1: Intuition for Mapping Reducibility

The condition $w \in A \iff f(w) \in B$ says two things:

- If $w \in A$, then $f(w) \in B$ ("yes" instances map to "yes" instances).
- If $w \notin A$, then $f(w) \notin B$ ("no" instances map to "no" instances).

So f perfectly preserves membership: it translates the question "is $w \in A$?" into the question "is $f(w) \in B$?" without changing the answer.

A mapping reduction of A to B provides a way to convert membership testing in A to membership testing in B. To test whether $w \in A$, we use the reduction f to map w to $f(w)$ and test whether $f(w) \in B$.

Remark 2.2: $\leq_{\mathrm{m}}$ is Reflexive and Transitive

The relation $\leq_{\mathrm{m}}$ is a **preorder**:

- **Reflexive:** $A \leq_{\mathrm{m}} A$ via the identity function $f(w) = w$.
- **Transitive:** If $A \leq_{\mathrm{m}} B$ via f and $B \leq_{\mathrm{m}} C$ via g, then $A \leq_{\mathrm{m}} C$ via $g \circ f$.

This justifies writing chains like $A \leq_{\mathrm{m}} B \leq_{\mathrm{m}} C$ and concluding $A \leq_{\mathrm{m}} C$.

2.3 The Reduction Theorem

The following theorem is the main tool for transferring classification results. It applies to all three classes R, RE, and co-RE — not just decidability.

Theorem 2.1: The Reduction Theorem (General Form)

Let $\mathcal{C} \in \{\mathrm{R}, \mathrm{RE}, \text{co-RE}\}$ be a class of languages, and let $A \leq_{\mathrm{m}} B$. If $B \in \mathcal{C}$, then $A \in \mathcal{C}$. Equivalently, if $A \notin \mathcal{C}$, then $B \notin \mathcal{C}$.

Proof. Let f be the reduction from A to B, so $w \in A \iff f(w) \in B$ for all w.

Case $\mathcal{C} = \mathrm{R}$: Let M be a decider for B. Build N: on input w, compute $f(w)$, then run M on $f(w)$ and output whatever M outputs. Since f always halts (computable) and M always halts (decider), N always halts. And N accepts w iff M accepts $f(w)$ iff $f(w) \in B$ iff $w \in A$. So N decides A.

Case $\mathcal{C} = \mathrm{RE}$: Let M be a recognizer for B. Build N: on input w, compute $f(w)$, then run M on $f(w)$. If M accepts, accept. Then N accepts w iff M accepts $f(w)$ iff $f(w) \in B$ iff $w \in A$. So N recognizes A.

Case $\mathcal{C} = \text{co-RE}$: If $B \in \text{co-RE}$, then $\overline{B} \in \mathrm{RE}$. Since $w \in A \iff f(w) \in B$, we also have $w \notin A \iff f(w) \notin B$, i.e., $w \in \overline{A} \iff f(w) \in \overline{B}$. So f is also a reduction from $\overline{A}$ to $\overline{B}$. By the RE case, $\overline{A} \in \mathrm{RE}$, hence $A \in \text{co-RE}$. □

Corollary 2.1: Undecidability Transfers Upward

If $A \leq_{\mathrm{m}} B$ and A is undecidable, then B is undecidable.

Proof. Contrapositive of the R case: if B were decidable, then A would be decidable. □

Remark 2.3: Direction of the Reduction

A common source of confusion: to prove that B is undecidable (or not in RE, etc.), we reduce *from* a known undecidable language A *to* B, i.e., we show $A \leq_{\mathrm{m}} B$. The reduction goes from the **known** problem to the **target** problem.

2.4 The Complement Trick

Remark 2.4: The Complement Trick

A reduction from A to B is automatically a reduction from $\overline{A}$ to $\overline{B}$. This is because $w \in A \iff f(w) \in B$ is logically equivalent to $w \notin A \iff f(w) \notin B$, i.e., $w \in \overline{A} \iff f(w) \in \overline{B}$.
This is extremely useful: a **single** computable function f gives you **two** reductions for the price of one.

2.5 The Three-Part Proof Template

The recitation formalizes mapping reduction proofs into a clean three-part structure:

(I) **Construction:** Define the computable function f. Typically $f(\langle M, w\rangle) = \langle M'\rangle$ where M' is a helper machine whose behavior depends on whether M accepts w.

(II) **Correctness:** Verify the "if and only if":

- $\langle M, w\rangle \in A \implies f(\langle M, w\rangle) \in B$
- $\langle M, w\rangle \notin A \implies f(\langle M, w\rangle) \notin B$

(III) **Computability:** Argue that a TM can compute f. Usually this amounts to saying that M' can be constructed by hard-coding M and w into its description.

Remark 2.5: Handling Invalid Inputs

Technically, f must be defined on *all* inputs $w \in \Sigma^*$, not just valid encodings of the form $\langle M, w\rangle$. If the input is not a valid encoding, we can define $f(w)$ to be any fixed string that is *not* in the target language B (e.g., the empty string ε). In practice, we ignore this case, but it is required for full rigor.

3 Language Classification Examples

Every language falls into exactly one of four categories:

$$\mathrm{R}, \qquad \mathrm{RE} \setminus \mathrm{R}, \qquad \text{co-RE} \setminus \mathrm{R}, \qquad \overline{\mathrm{RE} \cup \text{co-RE}}$$

Armed with the reduction theorem and the three-part proof template, we now classify a series of languages. We begin with a decidable example, then work through languages in RE $\setminus$ R, co-RE $\setminus$ R, and finally languages outside RE $\cup$ co-RE entirely.

3.1 A Decidable Example: Time-Bounded Machines

Not every language about TMs is undecidable. If the question involves a **bounded** computation, it is often decidable.

Definition 3.1: Time-Bounded Rejection

$$L = \{\langle M, w\rangle \mid M \text{ rejects } w \text{ within } |w|^3 + 10 \text{ steps}\}$$

Theorem 3.1: $L \in \mathrm{R}$

L is decidable.

Proof. Define a TM T as follows:

Decider T for L

T = "On input $\langle M, w\rangle$:
1. Simulate M on w for at most $|w|^3 + 10$ steps.
2. If M rejected during the simulation, **accept**.
3. Otherwise (if M accepted or did not halt within the limit), **reject**."

T always halts because the simulation is bounded by $|w|^3 + 10$ steps, and T accepts exactly when M rejects w within the time limit. Therefore T decides L. □

3.2 NON-EMPTY$_{\text{TM}} \in$ RE $\setminus$ R

Definition 3.2: NON-EMPTY$_{\text{TM}}$

$$\text{NON-EMPTY}_{\text{TM}} = \{\langle M\rangle \mid M \text{ is a TM and } L(M) \neq \emptyset\}$$

We already know from previous work that NON-EMPTY$_{\text{TM}} \in$ RE (enumerate all strings and simulate M on each one in a dovetailing fashion; accept if any simulation accepts). We now show it is not decidable.

Theorem 3.2: NON-EMPTY$_{\text{TM}} \in$ RE $\setminus$ R

NON-EMPTY$_{\text{TM}}$ is RE but not decidable.

Proof. We show $A_{\text{TM}} \leq_{\text{m}}$ NON-EMPTY$_{\text{TM}}$. Since $A_{\text{TM}} \notin$ R, it follows that NON-EMPTY$_{\text{TM}} \notin$ R.

(I) Construction: Define $f(\langle M, w\rangle) = \langle M'\rangle$, where M' operates as follows on input $x \in \Sigma^*$:

TM M' (erase-and-run)

M' = "On input x:
1. Erase the input x.
2. Write w on the tape.
3. Run M on w. If M halts, give the same answer."

(II) Correctness:

- If $\langle M, w\rangle \in A_{\text{TM}}$: then M accepts w, so M' accepts every input x. In particular $L(M') \neq \emptyset$, so $\langle M'\rangle \in$ NON-EMPTY$_{\text{TM}}$. ✓
- If $\langle M, w\rangle \notin A_{\text{TM}}$: then M does not accept w, so M' does not accept any input x. Hence $L(M') = \emptyset$, so $\langle M'\rangle \notin$ NON-EMPTY$_{\text{TM}}$. ✓

(III) Computability: A TM can compute f by outputting the encoding $\langle M'\rangle$, which hard-codes both w and M into M'. The machine M' first erases the input, writes w, then passes control to M. □

Remark 3.1: The "Erase-and-Run" Pattern

Notice that M' completely **ignores** its input x — it erases it and runs M on the fixed string w instead. This means:
- If M accepts w: M' accepts *everything* ($L(M') = \Sigma^*$).
- If M doesn't accept w: M' accepts *nothing* ($L(M') = \emptyset$).

This "erase-and-run" trick is the most common helper machine construction. It converts the question "does M accept w?" into an all-or-nothing language property. We will reuse it several times below.

3.3 $\text{EMPTY}_{\text{TM}} \in \text{co-RE} \setminus \text{R}$

Definition 3.3: EMPTY_{TM}

$$\text{EMPTY}_{\text{TM}} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset\}$$

Note: EMPTY_{TM} is essentially $\overline{\text{NON-EMPTY}_{\text{TM}}}$ (up to handling of invalid encodings).

Theorem 3.3: $\text{EMPTY}_{\text{TM}} \in \text{co-RE} \setminus \text{R}$

EMPTY_{TM} is co-RE but not decidable.

Proof. By the complement trick, the reduction $A_{\text{TM}} \leq_{\text{m}} \text{NON-EMPTY}_{\text{TM}}$ from the previous proof is also a reduction $\overline{A_{\text{TM}}} \leq_{\text{m}} \overline{\text{NON-EMPTY}_{\text{TM}}}$. Since $\overline{\text{NON-EMPTY}_{\text{TM}}}$ is essentially EMPTY_{TM}, we get $\overline{A_{\text{TM}}} \leq_{\text{m}} \text{EMPTY}_{\text{TM}}$.

Now:

- Since $A_{\text{TM}} \notin \text{R}$ and $A_{\text{TM}} \leq_{\text{m}} \text{NON-EMPTY}_{\text{TM}}$, we have $\text{NON-EMPTY}_{\text{TM}} \notin \text{R}$, hence $\text{EMPTY}_{\text{TM}} \notin \text{R}$.
- Since $\overline{A_{\text{TM}}} \notin \text{RE}$ and $\overline{A_{\text{TM}}} \leq_{\text{m}} \text{EMPTY}_{\text{TM}}$, we have $\text{EMPTY}_{\text{TM}} \notin \text{RE}$.
- Since $\text{NON-EMPTY}_{\text{TM}} \in \text{RE}$, we have $\text{EMPTY}_{\text{TM}} \in \text{co-RE}$.

Therefore $\text{EMPTY}_{\text{TM}} \in \text{co-RE} \setminus \text{R}$. □

3.4 $\text{ALL}_{\text{TM}} \notin \text{RE} \cup \text{co-RE}$

We now move to languages that lie completely outside $\text{RE} \cup \text{co-RE}$.

Definition 3.4: ALL_{TM}

$$\text{ALL}_{\text{TM}} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \Sigma^*\}$$

Theorem 3.4: $\text{ALL}_{\text{TM}} \notin \text{RE} \cup \text{co-RE}$

ALL_{TM} is neither RE nor co-RE.

We prove each direction separately. The co-RE direction uses the familiar erase-and-run pattern; the RE direction requires a new trick.

Part 1: $\text{ALL}_{\text{TM}} \notin \text{co-RE}$

Proof. We show $A_{\text{TM}} \leq_{\text{m}} \text{ALL}_{\text{TM}}$. Since $A_{\text{TM}} \notin \text{co-RE}$, it follows that $\text{ALL}_{\text{TM}} \notin \text{co-RE}$.

(I) Construction: Define $f(\langle M, w \rangle) = \langle M' \rangle$ using erase-and-run:

TM M'

M' = "On input x:
1. Erase the input x. Write w on the tape.
2. Run M on w. If M halts, give the same answer."

(II) Correctness:

- If $\langle M, w \rangle \in A_{\text{TM}}$: M accepts w, so M' accepts every x, hence $L(M') = \Sigma^*$ and $\langle M' \rangle \in \text{ALL}_{\text{TM}}$. ✓
- If $\langle M, w \rangle \notin A_{\text{TM}}$: M does not accept w, so M' does not accept any x, hence $L(M') = \emptyset \neq \Sigma^*$ and $\langle M' \rangle \notin \text{ALL}_{\text{TM}}$. ✓

(III) Computability: Hard-code M and w into M'. □

Part 2: $\mathrm{ALL}_{\mathrm{TM}} \notin \mathrm{RE}$

To show $\mathrm{ALL}_{\mathrm{TM}} \notin \mathrm{RE}$, we need a reduction from $\overline{A_{\mathrm{TM}}}$ (which is the language not in RE). The erase-and-run pattern does not work here — it gives a reduction from A_{TM}, not $\overline{A_{\mathrm{TM}}}$. We need a new construction.

Proof. We show $\overline{A_{\mathrm{TM}}} \leq_{\mathrm{m}} \mathrm{ALL}_{\mathrm{TM}}$. Since $\overline{A_{\mathrm{TM}}} \notin \mathrm{RE}$, it follows that $\mathrm{ALL}_{\mathrm{TM}} \notin \mathrm{RE}$.

(I) Construction: Define $f(\langle M, w\rangle) = \langle M'\rangle$ where:

TM M' (the step-counting trick)

M' = "On input x:
1. Run M on w for $|x|$ steps.
2. If M accepted during the run, **reject**. Otherwise, **accept**."

(II) Correctness:

- If $\langle M, w\rangle \in \overline{A_{\mathrm{TM}}}$ (i.e., M does *not* accept w): For every $x \in \Sigma^*$, running M on w for $|x|$ steps never results in acceptance. So M' accepts every x, hence $L(M') = \Sigma^*$ and $\langle M'\rangle \in \mathrm{ALL}_{\mathrm{TM}}$. ✓
- If $\langle M, w\rangle \notin \overline{A_{\mathrm{TM}}}$ (i.e., M *does* accept w): Let t be the number of steps M takes to accept w. For any x with $|x| \geq t$, the simulation of M on w for $|x|$ steps will reach acceptance, so M' rejects such x. Hence $L(M') \neq \Sigma^*$ and $\langle M'\rangle \notin \mathrm{ALL}_{\mathrm{TM}}$. ✓

(III) Computability: A TM can compute f by hard-coding M and w into M'. The machine M' additionally counts the input length and runs M with a step counter. □

Remark 3.2: The Step-Counting Trick

This construction is fundamentally different from erase-and-run. Here, M' *does* use its input x — not for its content, but for its **length**. The length $|x|$ serves as a "clock" that determines how many steps of M's computation on w to simulate.
This trick works because:
- If M never accepts w: no amount of time is enough, so M' accepts all inputs.
- If M accepts w in t steps: inputs of length $\geq t$ will "see" the acceptance, and M' rejects them.

The result is that $L(M') = \Sigma^*$ when M doesn't accept w, and $L(M')$ is a *finite* language (strings shorter than t) when M does accept w.

3.5 $\mathrm{EQ}_{\mathrm{TM}}$: Undecidable and Neither RE nor co-RE

We collect all results about $\mathrm{EQ}_{\mathrm{TM}}$ in one place. First we show it is undecidable (by reducing from E_{TM}), then we strengthen this to show it lies entirely outside RE ∪ co-RE.

Definition 3.5: $\mathrm{EQ}_{\mathrm{TM}}$

$$\mathrm{EQ}_{\mathrm{TM}} = \{\langle M_1, M_2\rangle \mid M_1 \text{ and } M_2 \text{ are TMs and } L(M_1) = L(M_2)\}$$

$\mathrm{EQ}_{\mathrm{TM}}$ **is Undecidable**

Theorem 3.5: $\mathrm{EQ}_{\mathrm{TM}}$ **is Undecidable**

$\mathrm{EQ}_{\mathrm{TM}}$ is undecidable.

The key observation is that E_{TM} is a special case of $\mathrm{EQ}_{\mathrm{TM}}$: testing whether $L(M) = \emptyset$ is the same as testing whether $L(M) = L(M_\emptyset)$, where $M_\emptyset$ is a TM that rejects all inputs.

Proof. Assume that TM R decides $\mathrm{EQ}_{\mathrm{TM}}$. We construct TM S that decides E_{TM}.

TM S deciding E_{TM} (assuming R decides EQ_{TM})

S = "On input $\langle M \rangle$, where M is a TM:
1. Construct a TM $M_\emptyset$ that rejects every input.
2. Run R on input $\langle M, M_\emptyset \rangle$.
3. If R accepts (i.e., $L(M) = L(M_\emptyset) = \emptyset$), **accept**.
4. If R rejects (i.e., $L(M) \neq \emptyset$), **reject**."

If R decides EQ_{TM}, then S decides E_{TM}. Since E_{TM} is undecidable, the decider R for EQ_{TM} cannot exist. Therefore EQ_{TM} is undecidable. □

3.6 Every RE Language Reduces to A_{TM}

Theorem 3.6: $L \in \text{RE} \implies L \leq_{\text{m}} A_{\text{TM}}$

For every $L \in \text{RE}$, we have $L \leq_{\text{m}} A_{\text{TM}}$.

Proof. Let M be a TM that recognizes L.

(I) Construction: Define $f(w) = \langle M, w \rangle$.

(II) Correctness: $w \in L \iff M$ accepts $w \iff \langle M, w \rangle \in A_{\text{TM}}$.

(III) Computability: A TM can compute f by writing the (fixed) encoding $\langle M \rangle$ next to w. □

4 Exam-Style Examples

4.1 Machines That Accept Every Word of Some Length

Definition 4.1: $L_{\text{all-length}}$

$$L = \{\langle M \rangle \mid \exists\, n \geq 0 \text{ such that } M \text{ accepts every word of length } n\}$$

Theorem 4.1: $L \in \text{RE} \setminus \text{R}$

L is RE but not decidable.

$L \in \text{RE}$

Proof. We define a TM T that recognizes L:

TM T recognizing L

T = "On input $\langle M \rangle$:
1. For $i = 0, 1, 2, \ldots$:
   (a) For $n = 0, 1, 2, \ldots, i$:
      - For every word w of length n: run M on w for at most i steps.
      - If M did not accept some w within i steps, continue to the next n.
      - If M accepted *all* words of length n within i steps, **accept**.

Correctness ($L(T) = L$):

- If T accepts $\langle M \rangle$: then some n was found where M accepts every word of length n, so $\langle M \rangle \in L$.
- If $\langle M \rangle \in L$: let n' be a length where M accepts all words of that length, and let t be the maximum number of steps M takes on any word of length n'. When T reaches iteration $i = \max(n', t)$ and within it $n = n'$, it discovers that M accepts all words of length n' within $t \leq i$ steps, so T accepts.

Therefore T recognizes L, so $L \in \text{RE}$. □

$L \notin \mathrm{R}$

Proof. We show $A_{\mathrm{TM}} \leq_{\mathrm{m}} L$. Since $A_{\mathrm{TM}} \notin \mathrm{R}$, it follows that $L \notin \mathrm{R}$.

(I) Construction: Define $f(\langle M, w\rangle) = \langle M'\rangle$ using erase-and-run:

TM M'

$M' =$ "On input x:
1. Erase the input x. Write w on the tape.
2. Run M on w. If M halts, give the same answer."

(II) Correctness:

- If $\langle M, w\rangle \in A_{\mathrm{TM}}$: M accepts w, so M' accepts every x. In particular, M' accepts every word of every length, so $\langle M'\rangle \in L$. ✓
- If $\langle M, w\rangle \notin A_{\mathrm{TM}}$: M does not accept w, so M' accepts nothing. There is no n for which M' accepts every word of length n, so $\langle M'\rangle \notin L$. ✓

(III) Computability: Hard-code M and w into M'. □

4.2 Machines with Reverse-Equal Languages

Definition 4.2: L_{rev}

$$L = \{\langle M_1, M_2\rangle \mid L(M_1) = L(M_2)^{\mathrm{rev}}\}$$

where $L^{\mathrm{rev}} = \{w^{\mathrm{rev}} \mid w \in L\}$.

Theorem 4.2: $L \notin \mathrm{RE} \cup \text{co-RE}$

L is neither RE nor co-RE.

Proof. We show $\mathrm{ALL}_{\mathrm{TM}} \leq_{\mathrm{m}} L$. Since $\mathrm{ALL}_{\mathrm{TM}} \notin \mathrm{RE}$ and $\mathrm{ALL}_{\mathrm{TM}} \notin \text{co-RE}$, it follows by the reduction theorem that $L \notin \mathrm{RE}$ and $L \notin \text{co-RE}$.

(I) Construction: Define $f(\langle M\rangle) = \langle M, T_{\mathrm{all}}\rangle$, where T_{all} is a TM that immediately accepts every input (i.e., $q_0 = q_{\mathrm{acc}}$).

(II) Correctness: Since every word $w \in \Sigma^*$ can be written as $(w^{\mathrm{rev}})^{\mathrm{rev}}$, we have $(\Sigma^*)^{\mathrm{rev}} = \Sigma^*$. Also $L(T_{\mathrm{all}}) = \Sigma^*$, so $L(T_{\mathrm{all}})^{\mathrm{rev}} = \Sigma^*$. Therefore:

$$\langle M\rangle \in \mathrm{ALL}_{\mathrm{TM}} \iff L(M) = \Sigma^* \iff L(M) = L(T_{\mathrm{all}})^{\mathrm{rev}} \iff \langle M, T_{\mathrm{all}}\rangle \in L$$

(III) Computability: A TM can output $\langle M, T_{\mathrm{all}}\rangle$ given $\langle M\rangle$ by encoding the constant $\langle T_{\mathrm{all}}\rangle$ next to $\langle M\rangle$. □

5 Summary

5.1 Classification Table

Language	**Classification**	**Key Reduction**
$\{\langle M, w\rangle \mid M \text{ rejects } w \text{ within } \lvert w\rvert^3 + 10 \text{ steps}\}$	R	(direct simulation with bound)
$A_{\mathrm{TM}} = \{\langle M, w\rangle \mid M \text{ accepts } w\}$	$\mathrm{RE} \setminus \mathrm{R}$	(diagonal argument)
$\text{NON-EMPTY}_{\mathrm{TM}} = \{\langle M\rangle \mid L(M) \neq \emptyset\}$	$\mathrm{RE} \setminus \mathrm{R}$	$A_{\mathrm{TM}} \leq_{\mathrm{m}} \text{NON-EMPTY}_{\mathrm{TM}}$
$L_{\text{all-length}}$	$\mathrm{RE} \setminus \mathrm{R}$	$A_{\mathrm{TM}} \leq_{\mathrm{m}} L$
$\overline{A_{\mathrm{TM}}}$	$\text{co-RE} \setminus \mathrm{R}$	complement of A_{TM}
$\mathrm{EQ}_{\mathrm{TM}} = \{\langle M_1, M_2\rangle \mid L(M_1) = L(M_2)\}$	$\notin \mathrm{RE} \cup \text{co-RE}$	$A_{\mathrm{TM}} \leq_{\mathrm{m}} \overline{\mathrm{EQ}_{\mathrm{TM}}}$, complement trick
$\mathrm{ALL}_{\mathrm{TM}} = \{\langle M\rangle \mid L(M) = \Sigma^*\}$	$\notin \mathrm{RE} \cup \text{co-RE}$	erase-and-run + step-counting
$L_{\mathrm{rev}} = \{\langle M_1, M_2\rangle \mid L(M_1) = L(M_2)^{\mathrm{rev}}\}$	$\notin \mathrm{RE} \cup \text{co-RE}$	$\mathrm{ALL}_{\mathrm{TM}} \leq_{\mathrm{m}} L_{\mathrm{rev}}$

5.2 Reduction Patterns

Three main helper machine patterns appear repeatedly:

1. **Erase-and-run:** M' ignores its input, runs M on the fixed w. Produces $L(M') = \Sigma^*$ or $L(M') = \emptyset$. Used for: $\text{NON-EMPTY}_{\mathrm{TM}}$, $\mathrm{ALL}_{\mathrm{TM}}$ (co-RE direction), $\mathrm{EQ}_{\mathrm{TM}}$, $L_{\text{all-length}}$.
2. **Filter-then-run:** M' first checks if x has some property (e.g., $x = w$, or $x \in \{0^n 1^n\}$), then conditionally runs M on w. Produces languages that toggle between a specific set and $\emptyset$ or Σ^*. Used for: E_{TM}, $\mathrm{REGULAR}_{\mathrm{TM}}$.
3. **Step-counting:** M' runs M on w for $|x|$ steps, using the input length as a clock. Produces $L(M') = \Sigma^*$ or a finite language. Used for: $\mathrm{ALL}_{\mathrm{TM}}$ (RE direction).